

Lecture 8 — 10/09

Lecturer: Arjun Bhagoji

Scribe: Aryan Katiyar

8.1 Poison attacks

We start off by first understanding what poisoning attacks are. Poisoning is basically tampering with the training data, test data(or both), or the parameters of an ML model to observe or exploit the vulnerability opened at the test time. In a survey [3] of 28 organizations spanning sectors such as cybersecurity, healthcare, and government, data poisoning was identified as the primary adversarial threat; out of the 28 respondents, 10 selected poisoning as the single attack that would most affect their business operations.

8.1.1 Flavours of poisoning attack

The main goal of a poisoning attack is to undermine a model's reliability. A compromised model will perform correctly on most inputs but is engineered to fail on specific inputs chosen by the attacker, forcing it to produce either a generally incorrect prediction or a specific output desired by the adversary.

- **Perturbing Model Parameters (Model Poisoning):** This attack involves directly tampering with the model's learned parameters, h , after the training process. The objective is to corrupt the model's behavior without altering the data itself.
- **Perturbing Training Data (D_{train}):** Known as data poisoning, this attack corrupts the training set to compromise the learning process and embed vulnerabilities in the resulting model. These attacks can be further classified based on the attacker's capability to modify input features, labels, or both, as we will see in detail later.
- **Perturbing Test Data (D_{test}):** Also called Evasion Attacks, these attacks target fully trained models at inference time. In this approach, an attacker crafts adversarial examples by applying small, often imperceptible, perturbations to an input, $\vec{x}$, with the intent to cause a misclassification. In this setting we typically assume that the Training data (D_{train}) wasn't tampered with which gives us the Distributionally Robust Optimization (DRO) setting.

8.2 Threat Modeling

To understand poisoning attacks effectively, it is essential to first define the ultimate objective of the attacker and assess the attacker's capabilities. The goal of the attacker generally revolves around two main factors:

1. The desired **outcome** of the attack, i.e., what the attacker intends to achieve by tampering with the training process.
2. The **capabilities** of the attacker, which determine the feasibility and strength of the attack. The following subsections describe these factors in detail.

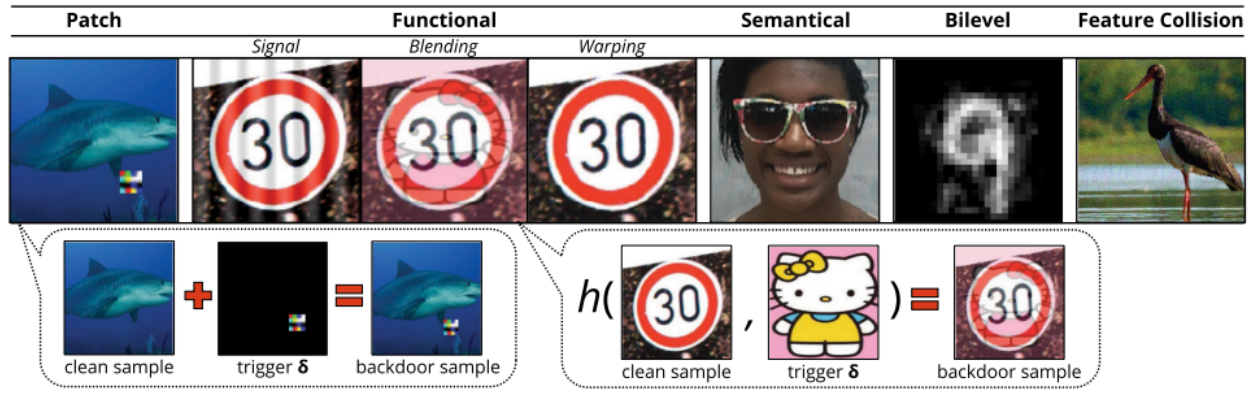


Figure 8.1. Examples of adversarial perturbations in poisoning attacks, categorized by type. The top row illustrates different trigger designs for backdoor attacks, including **Patch**, **Functional** (Signal, Blending, Warping), and **Semantical** triggers. The bottom row visually explains how a clean sample is combined with a trigger to form a backdoor sample. The images under **Bilevel** and **Feature Collision** demonstrate techniques related to clean-label poisoning, where subtle, often imperceptible, feature manipulations aim to induce misclassification without altering the label [2].

8.2.1 Attacker Aims

The attacker's aim in poisoning attacks typically falls into two broad categories:

- **Indiscriminate Poisoning Attack (Availability):** The attacker attempts to corrupt the training dataset so that the model becomes unreliable or even useless during test time. This type of attack decreases the overall accuracy of the model and compromises its availability for legitimate users.

Attacker generates $\tilde{D}_{\text{train}} = \mathcal{A}(D_{\text{train}})$ such that $\mathbb{E}_{z \sim P^*}[\ell(z, \hat{h})]$ is maximized,

where $\hat{h} \in \arg \min_{h \in \mathcal{H}} L(\tilde{D}_{\text{train}}, h)$ and $TV(D_{\text{train}}, \tilde{D}_{\text{train}}) \leq \delta$.

- **Targeted Poisoning Attacks(Integrity):** In this scenario, the attacker focuses on obtaining specific, targeted misclassifications for a chosen set of inputs, while the model continues to perform well on benign, clean data. These attacks are harder to detect and are considered more dangerous because they directly challenge the *integrity* of the model. These can be further categorized.

- **Without triggers:** Generate $\tilde{D}_{\text{train}} = A(D_{\text{train}})$ such that either
 - i) $E_{z \sim P_t^*}[L(z, \tilde{h})]$ is maximized OR ii) $E_{z=(\vec{x}, \vec{y}) \sim P_t^*}[L((\vec{x}, \vec{t}), \tilde{h})]$ is minimized,
 while $E_{z \sim P^*/P_t^*}[L(z, \tilde{h})]$ is minimized, where $\tilde{h} \in \operatorname{argmin}_{h \in \mathcal{H}} \hat{L}(\tilde{D}_{\text{train}}, h)$. Here P_t^* represents the distribution of the target samples the attacker wishes to poison.
- **With triggers:** Generate trigger $\vec{\tau}$ per sample such that either
 - i) $E_{z_t \sim P_t^*}[L((z_t + \vec{\tau}, y_t), \tilde{h})]$ is maximized OR ii) $E_{z_t \sim D_t}[L((z_t + \vec{\tau}, T), \tilde{h})]$ is minimized
 where $\tilde{h} \in \operatorname{argmin}_{h \in \mathcal{H}} \hat{L}(\tilde{D}_{\text{train}}, h)$ and δ -fraction of $\tilde{D}_{\text{train}}$ has triggers.

Indiscriminate poisoning attacks are easier to detect since they degrade overall performance. In contrast, targeted attacks are stealthier and often remain hidden during validation and deployment, making them particularly concerning.

8.2.2 Attacker's Knowledge

The effectiveness and design of an attack depend heavily on the attacker's knowledge of the target system. This knowledge can span three main components: (i) the **training data** (D_{train}), (ii) the **machine learning model**, including its architecture ($\mathcal{H}$), trained parameters (θ), and learning algorithm, and (iii) the **test data** (D_{test}). Based on the extent of this knowledge, we can define two primary attack settings.

- **White-box Setting** In a white-box attack, the adversary is assumed to have complete knowledge and access to the target system. This represents a worst-case scenario and is crucial for robustly evaluating defensive strategies. This knowledge includes:
 - The training dataset (D_{train}).
 - The model's architecture or hypothesis space ($\mathcal{H}$).
 - Trained internal parameters (θ), and the training algorithm.
 - The loss function (l) used for training.
 - The testset distribution (P^*) that will be used for evaluation.
- **Black-box Setting** In a black-box setting, the attacker has limited or no access to the model's internal workings or the training data. The model is effectively an opaque box that only provides outputs for given inputs. Attacks in this setting are more practical and are typically staged in one of two ways:
 - **Transfer-based Attacks:** The attacker leverages partial knowledge to build a local *surrogate* or *proxy* model. They collect a surrogate dataset ($D_{\text{train}}^{\text{proxy}}$) to train their own model ($\mathcal{H}^{\text{proxy}}$) that mimics the behavior of the target model by construction their own loss functions (l^{proxy}). The attacker then performs a white-box attack on this surrogate model and transfers the crafted malicious inputs (e.g., poisoned data or adversarial examples) to the target, assuming they will have a similar effect.
 - **Query-based Attacks:** The attacker interacts with the target model by repeatedly sending it input queries and observing the corresponding predictions. By analyzing these input-output pairs, the attacker can iteratively refine an attack or infer decision boundaries of the model without any other knowledge. This approach treats the target model as an oracle.

8.2.3 Attacker Data Manipulation Capabilities

An attacker's ability to poison a dataset is defined by how they manipulate the training samples. The primary capabilities are:

- **Dirty-label Attacks:** The attacker modifies the labels of a fraction of the training data. This involves inspecting the training data and, for a sample z_i in the poisoned set D_p , replacing its true label y_i with a malicious one $\tilde{y}_i$. If only the labels are changed, this is often called a label-flip attack.
- **Clean-label Attacks:** The attacker applies small, often imperceptible perturbations to the features of the training samples ($\vec{x}_i \rightarrow \tilde{\vec{x}}_i$) while keeping their original, correct labels. This approach is significantly stealthier as the data appears benign.

- **Backdoor Attacks:** This is a specialized attack where the attacker modifies both features and labels. A specific pattern, the *trigger* ($\vec{\tau}$), is embedded into the features of some training samples ($\vec{x}_i \rightarrow \vec{x}_i + \vec{\tau}_i$), and their labels are simultaneously changed to a target class ($y_i \rightarrow t_i$). This creates a hidden vulnerability that must be activated at test time by adding the same trigger to test inputs.

8.3 Attacking Strategy

8.3.1 Label Flipping

Label flipping is a straightforward poisoning strategy where an adversary mislabels a subset of the training data without perturbing the feature values. This dirty-label attack is a method for indiscriminate poisoning that requires data access but not necessarily model access.

The primary approaches to performing a label-flipping attack include:

- **Random Label Flipping:** In this simple approach, the attacker randomly flips the labels of some training samples. This can be either untargeted (to degrade overall performance) or targeted. However, as noted in research [4], random flips often result in far-from-optimal attack performance.
- **Optimal Label Flipping:** This involves finding the specific set of label flips that will maximally degrade the model's performance.
 - **Untargeted:** Solving for the optimal flips is an NP-hard optimization problem, even for simple models like binary linear classifiers.
 - **Targeted:** The complexity for targeted optimal flipping remains an open research question.
- **Heuristic-based Flipping:** Due to the intractability of finding the optimal solution, heuristic strategies are used to efficiently approximate it.
 - **Untargeted:** Various heuristic algorithms have been proposed, such as the one by Biggio, Nelson, and Laskov (2011) for SVMs. [1] (HW: Understand Algorithm 1 in this paper)
 - **Targeted:** A common approach for targeted attacks involves using influence functions to identify which training points, if mislabeled, would have the most impact on a specific prediction.

Bibliography

- [1] Battista Biggio, Blaine Nelson, and Pavel Laskov. Poisoning attacks against support vector machines, 2013.
- [2] Antonio Emanuele Cinà, Kathrin Grosse, Ambra Demontis, Sebastiano Vascon, Werner Zellinger, Bernhard A. Moser, Alina Oprea, Battista Biggio, Marcello Pelillo, and Fabio Roli. Wild patterns reloaded: A survey of machine learning security against training data poisoning. *ACM Comput. Surv.*, 55(13s), July 2023.
- [3] Ram Shankar Siva Kumar, Magnus Nyström, John Lambert, Andrew Marshall, Mario Goertzel, Andi Comisssoneru, Matt Swann, and Sharon Xia. Adversarial machine learning - industry perspectives. *CoRR*, abs/2002.05646, 2020.
- [4] Huang Xiao, Battista Biggio, Blaine Nelson, Han Xiao, Claudia Eckert, and Fabio Roli. Support vector machines under adversarial label contamination. *Neurocomputing*, 160:53–62, July 2015.